

A&I

android and ios develop.

PyTorch

2026 A&I 4th

AI 멘토 구승민

목차

1. 학습법 안내

- 강의자료 및 실습 코드 확인 방법, AI 활용 학습법

2. Tensor 기초

- 벡터, 행렬, 텐서, 데이터타입, 텐서 변환 방법

3. 자동미분

- 경사 하강법, 연쇄 법칙, 손실 함수

4. 다층 퍼셉트론(MLP)

- 기울기 소실 문제, 활성화 함수, 옵티마이저

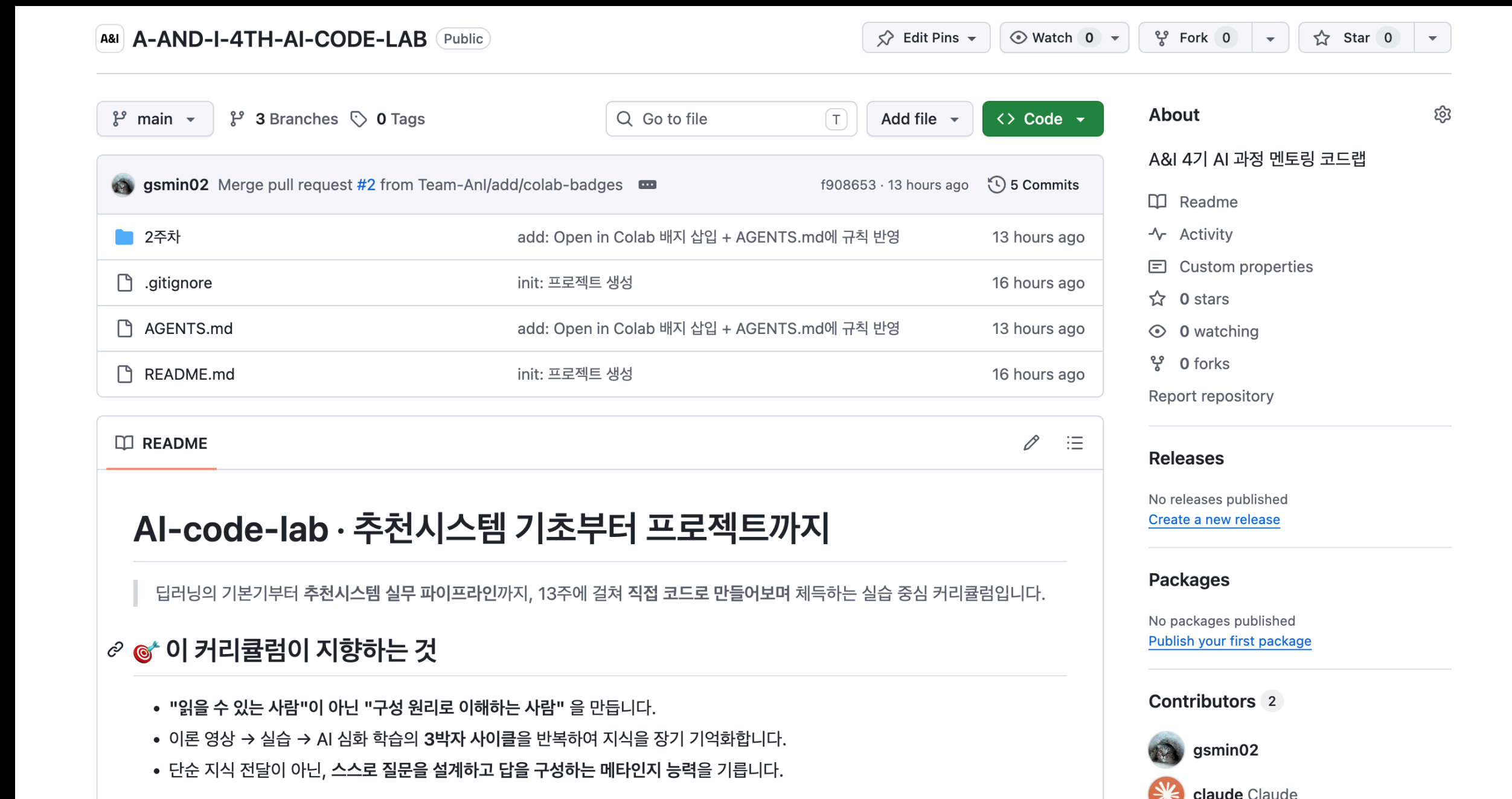
1. 학습법 안내

- 기본적인 강의 내용 확인 방법
- AI 활용 안내 사항

강의자료 확인 방법

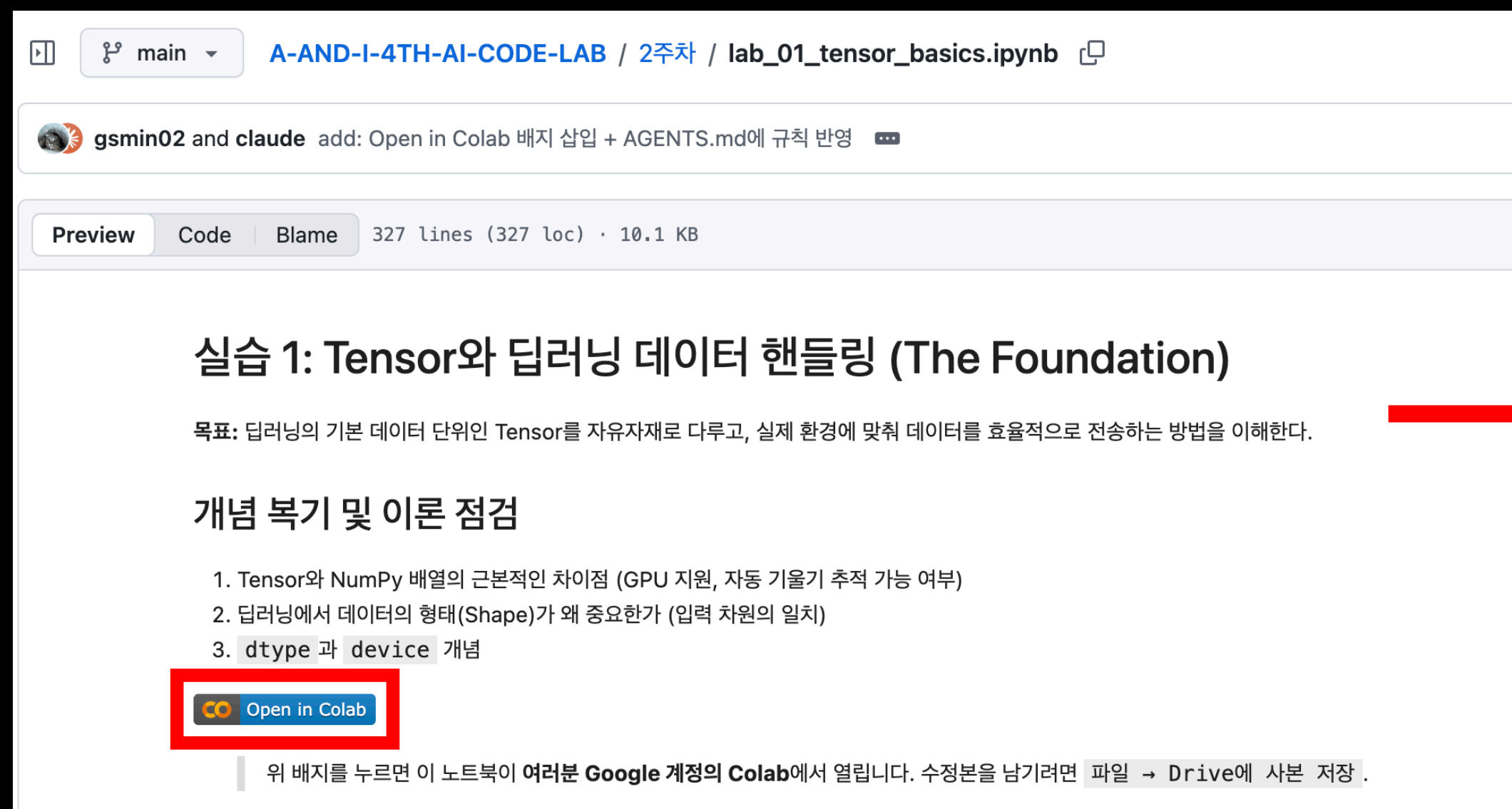
강의자료는 아래 링크를 통해 확인 가능

- <https://github.com/Team-Anl/A-AND-I-4TH-AI-CODE-LAB>

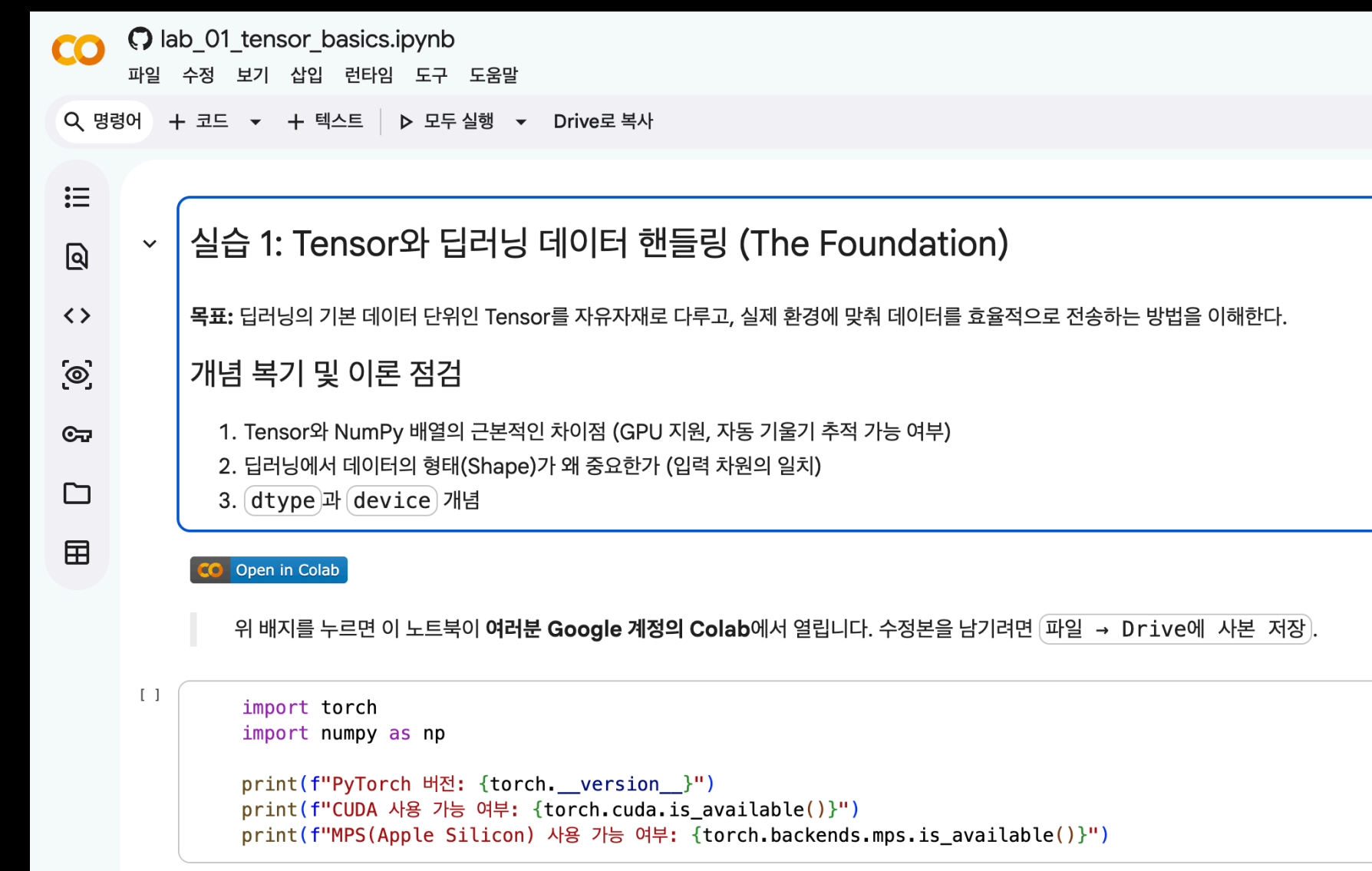


코드 실습 수행

각 실습은 제공된 ipynb 파일 상단 “**Open in Colab**”으로 실습 가능



The screenshot shows a GitHub repository page for the file 'lab_01_tensor_basics.ipynb'. The file is 327 lines (327 loc) and 10.1 KB. The 'Preview' tab is selected, showing the title '실습 1: Tensor와 딥러닝 데이터 핸들링 (The Foundation)'. Below the title is a description: '목표: 딥러닝의 기본 데이터 단위인 Tensor를 자유자재로 다루고, 실제 환경에 맞춰 데이터를 효율적으로 전송하는 방법을 이해한다.' followed by a section '개념 복기 및 이론 점검' with three numbered items. At the bottom, there is a red box highlighting the 'Open in Colab' button.

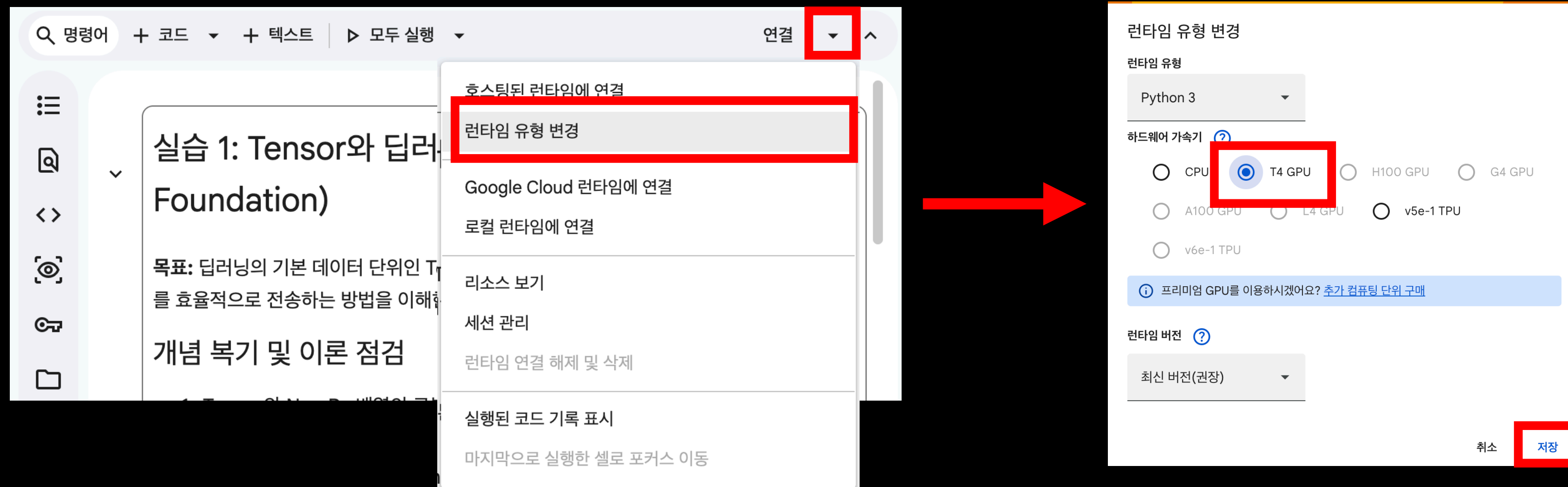


The screenshot shows the Google Colab interface for the file 'lab_01_tensor_basics.ipynb'. The title '실습 1: Tensor와 딥러닝 데이터 핸들링 (The Foundation)' is at the top. Below it is the same description and '개념 복기 및 이론 점검' section as in the GitHub preview. A red arrow points from the 'Open in Colab' button in the GitHub preview to the 'Open in Colab' button in the Colab interface. The Colab interface also shows the 'Open in Colab' button and a note: '위 배지를 누르면 이 노트북이 여러분 Google 계정의 Colab에서 열립니다. 수정본을 남기려면 파일 → Drive에 사본 저장.' Below this, there is a code cell with the following code:

```
[ ]  
import torch  
import numpy as np  
  
print(f"PyTorch 버전: {torch.__version__}")  
print(f"CUDA 사용 가능 여부: {torch.cuda.is_available()}")  
print(f"MPS(Apple Silicon) 사용 가능 여부: {torch.backends.mps.is_available()}")
```

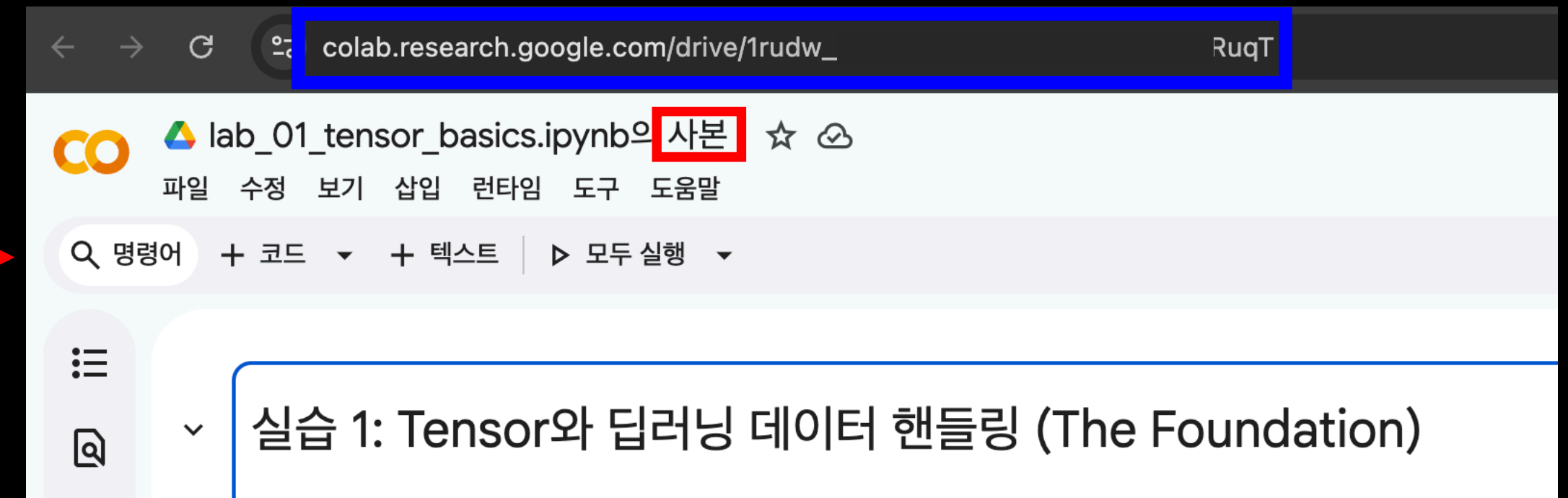
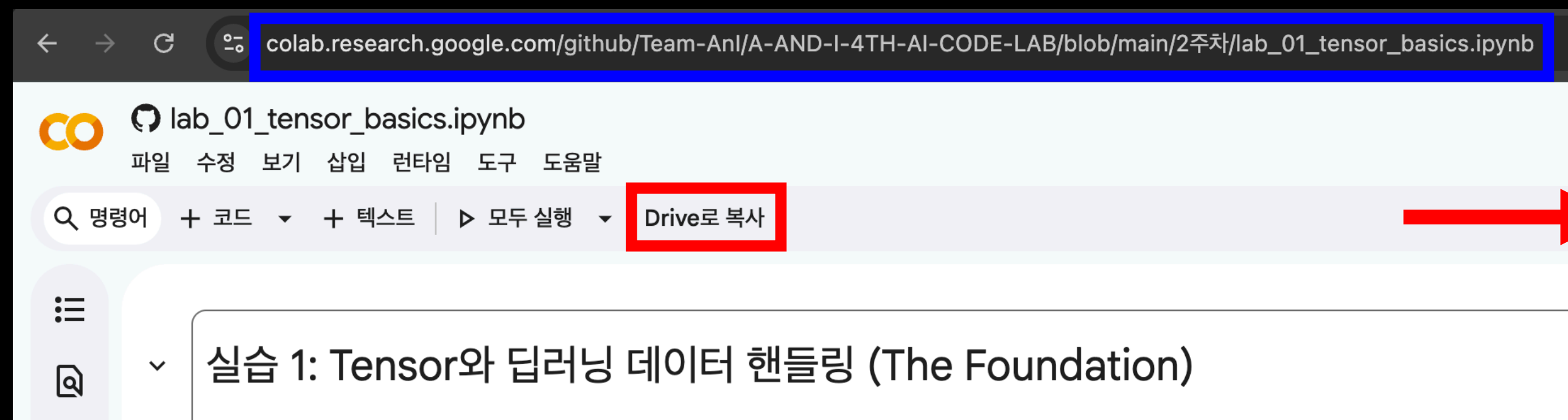
코드 실습 주의사항

실행 전 “런타임 유형 변경”을 통해 “T4 GPU”로 변경해야 함



내용 편집

내용을 변경하고 싶은 경우 “**Drive로 복사**”를 통해 편집 가능



AI 활용

AI 활용을 적극 권장합니다

- 다만, 그냥 물어보는 것이 아닌 “제대로” 물어보는 것이 중요합니다
- 학습에서 AI 활용은 단순히 묻기만 하면 제대로된 답변을 얻기 어렵습니다
- 자신이 알고 있는 지식이 어디까지인지 설명하고, 이해가 안되거나 부족한 부분에 대해 묻는 것이 맞습니다

안 좋은 예시

- “텐서가 뭔지 알려줘”

좋은 예시

- “벡터가 1차원의 정보, 행렬이 2차원의 정보를 표현하는 것은 알았어.
텐서는 3차원 이상을 표현한다는게 아직 이해가 안돼. 이 부분에 대해서 더 설명해줘.”

실습과 AI 질문하기

각 실습에는 AI에게 물어보기 좋은 내용을 구성했습니다

막혔을 때 — 3단계 탈출구

"모르는 건 당연합니다. 다만 AI에게 정답을 구걸하지 않고 빠져나오는 법을 익히세요."

Level 1 · 5분 이상 막혔다면 — 스스로 재확인

- 에러 메시지의 마지막 줄과 텐서의 `shape` / `dtype` / `device` 를 소리 내어 다시 읽기
- PyTorch 공식 문서에서 문제의 함수 한 줄 정의만 확인 (튜토리얼 말고 레퍼런스)

Level 2 · 10분 이상 막혔다면 — AI에게 "힌트만"

나는 PyTorch Tensor 실습 중이고, <내가 지금 만든 텐서 `shape/dtype/device` + 원하는 목표> 상태에서 막혔어. 정답 코드는 주지 말고, 다음 한 줄을 짜기 위해 내가 스스로에게 던져야 할 질문 하나만 알려줘.

Level 3 · 20분 이상 막혔다면 — 정답 받되 즉시 덮기

- AI에게 정답 코드를 받되, 창을 닫고 자기 손으로 재작성 → 그 뒤에 비교
- 달랐던 줄을 한 줄씩 말로 설명해 보기. 설명 못 하면 그 줄은 아직 모른다는 뜻.

AI 심화 학습 프롬프트

아래 프롬프트를 복사해서 ChatGPT / Claude 등에 붙여 넣어 보세요.

중요: 각 단계마다 AI의 답을 그대로 받아들이지 말고, "왜 그렇게 생각해?" 를 한 번 더 물어보세요.

Phase 1. 구조화 — 큰 그림부터 잡기

나는 딥러닝을 막 배우기 시작했어. 지금 "PyTorch의 Tensor"를 공부하려 해. 정답을 바로 알려주지 말고, Tensor라는 주제가 어떤 3~4개의 하위 축으로 구성되어 있을지 먼저 '지도'만 그려줘. 각 축이 왜 필요한지 한 문장씩 덧붙여줘. 그 뒤에, 내가 어떤 순서로 공부하면 좋을지도 제안해줘.

Phase 2. 개념 회상 — 내 요약을 검증받기

내가 방금 Tensor에 대해 이해한 내용을 아래처럼 정리했어. 이 설명에서 '빠진 핵심 개념' 또는 '틀린 부분'만 지적해줘. 답을 다시 써주지 말고, 질문 형식으로 내 사고의 빈틈만 찢어줘.

[여기에 내가 직접 쓴 3~5줄 요약 붙여넣기]

2. Tensor 기초

- 인공지능 정의
- 스칼라, 벡터, 행렬, 텐서
- 데이터타입
- 텐서 변환 방법

인공지능 정의

사람처럼 학습하고 추론하는 지능을 가진 시스템을 만드는 기술

인공지능
(Artificial Intelligence)

머신러닝
(Machine Learning)

딥러닝
(Deep Learning)

인공지능 정의

인공지능(Artificial Intelligence)

- 사람처럼 학습하고 추론할 수 있는 지능을 가진 시스템을 만드는 기술

머신러닝(Machine Learning)

- 규칙 선언 없이 자동으로 데이터에서 규칙을 학습하는 알고리즘

딥러닝(Deep Learning)

- 인공 신경망을 기반으로 한 방법들을 통칭하는 내용, 인공 신경망과 크게 구분하지 않음
- 인공 신경망(Artificial Neural Network)
 - 생물학적 뉴런에서 영감을 받아 만든 머신러닝 알고리즘, 종종 딥러닝이라고 불림

데이터의 표현

AI는 복잡한 문제를 해결하기 위해 고안되었습니다

많은 데이터를 활용하며, 이를 효율적으로 표현할 방법이 필요합니다

컴퓨터에서 이를 효과적으로 표현하는 방법은 텐서를 활용하는 것입니다

선형대수의 다차원 표현

스칼라(Scalar) / 0-D Tensor

- 1, 5, 1.2, -7 등 일반적인 수치로 하나의 숫자로 표현되는 양을 의미

벡터(Vector) / 1-D Tensor

- 벡터는 스칼라를 직선 상에 나열한 것으로 순서가 지정된 여러 개의 숫자들이 일렬로 나열된 구조를 의미

행렬(Matrix) / 2-D Tensor

- 스칼라를 격자 형태 또는 동일한 크기를 가진 1-D Tensor들이 모여서 형성한, 행과 열로 구성된 사각형 구조

텐서(Tensor) / 3-D Tensor / N-D Tensor ($N \geq 4$)

- 동일한 크기의 2-D Tensor들이 여러 개 쌓여 형성된 입체적인 배열 구조
- 동일한 크기의 (N-1)-D Tensor들이 여러 개 쌓여 형성된 입체적인 배열 구조

텐서 실습

텐서 기본 정의

- torch를 선언하여 해당 클래스 메서드로 호출 가능

List 활용 정의

- 파이썬 내부 자료형인 list를 통해 정의 가능

Numpy 활용 정의

- 수학 관련 라이브러리인 numpy의 배열로도 정의 가능

tuple 활용 정의

- 파이썬 내부 자료형인 tuple을 통해 정의 가능

```
import torch
import numpy as np

# (1) 리스트로부터 생성
t1 = torch.tensor([[1, 2, 3], [4, 5, 6]])

# (2) NumPy 배열로부터 생성
arr = np.array([[1.0, 2.0], [3.0, 4.0]])
t2 = torch.from_numpy(arr)

# (3) 0으로 채워진 텐서
t3 = torch.zeros((2, 3, 4))

# (4) 1로 채워진 텐서
t4 = torch.ones((3, 3))

# (5) 난수 텐서 (0~1 균등분포)
t5 = torch.rand((2, 2))

# (6) 정규분포 난수
t6 = torch.randn((2, 2))
```

텐서 실습

텐서의 형태

- 텐서 변수에 shape을 통해 확인 가능
- 나중에도 많이 활용하여 보는 방법을 잘 익히는 것이 중요
- 예) (2, 3, 4) 출력 시 **뒤에서부터 읽는 것을 추천**
- 4개의 스칼라가 1차원 벡터로 존재.
- 해당 벡터가 3개씩 있는 행렬로 존재.
- 해당 행렬이 2개가 있는 텐서로 존재.

```
for name, t in zip(["t1", "t2", "t3", "t4", "t5", "t6"], [t1, t2, t3, t4, t5, t6]):  
    print(f"{name}: shape={tuple(t.shape)}, dtype={t.dtype}")  
    print(t)  
    print("-" * 40)
```

```
t3: shape=(2, 3, 4), dtype=torch.float32  
tensor([[[0., 0., 0., 0.],  
         [0., 0., 0., 0.],  
         [0., 0., 0., 0.]],  
          
        [[0., 0., 0., 0.],  
         [0., 0., 0., 0.],  
         [0., 0., 0., 0.]])
```


텐서의 데이터 타입

자료형 선언 방식

- torch.int or torch.int64
- torch.float or torch.float32

자료형 변환

- 변수명.to(torch.float)
- 변수명.float()

```
# dtype 변환 실습
int_tensor = torch.tensor([1, 2, 3, 4])
print(f"원본: {int_tensor}, dtype={int_tensor.dtype}")

float_tensor = int_tensor.float() # float32로 변환
print(f"변환 후: {float_tensor}, dtype={float_tensor.dtype}")

double_tensor = int_tensor.to(torch.float64)
print(f".to() 변환: {double_tensor}, dtype={double_tensor.dtype}")
```

원본: tensor([1, 2, 3, 4]), dtype=torch.int64
변환 후: tensor([1., 2., 3., 4.]), dtype=torch.float32
.to() 변환: tensor([1., 2., 3., 4.]), dtype=torch.float64, dtype=torch.float64

서로 다른 dtype의 텐서끼리 연산 시 오류가 발생

텐서 변환 방법

인덱싱

- 특정 위치의 값을 단일 값으로 가져오는 방법

슬라이싱

- 특정 위치의 값을 범위의 형태로 가져오는 방법

view(), reshape()

- 텐서가 가지고 있는 형태(shape)를 변경하는 방법

특정 위치를 추출하는 방법

인덱싱

- 특정 위치의 값을 단일 값으로 가져오는 방법
- 변수명[값] 형태로 활용
- 인덱스는 0부터 시작

슬라이싱

- 특정 위치의 값을 범위의 형태로 가져오는 방법
- 변수명[시작:끝] 형태로 활용
- 단, 인덱스로 처리되어 끝 부분은 생략 됨
 - 예) [1:4] -> 1, 2, 3

```
# (1) 첫 번째 이미지만 뽑기
first = images[0]
print(f"첫 이미지 shape: {first.shape}")

# (2) 모든 이미지의 R채널만 뽑기
red_channel = images[:, 0, :, :]
print(f"R채널만 추출 shape: {red_channel.shape}")

# (3) 이미지의 가운데 16x16 영역만 잘라내기 (Center Crop)
center_crop = images[:, :, 8:24, 8:24]
print(f"Center crop shape: {center_crop.shape}")
```


텐서의 크기를 변환하는 방법

view()

- 텐서의 모양을 변경하는 방법으로 속도 빠름, 기존 공간을 변환하여 메모리를 재사용
- 데이터가 메모리에 연속적으로 된 경우만 가능

reshape()

- 텐서의 모양을 변경하는 방법으로 속도는 view보다 느림, 메모리에 새로 할당하여 정의
- 메모리가 연속이 아니어도 가능

```
# (4) Shape 변형 - view / reshape / flatten
flat = images.view(8, -1)
print(f"Flatten shape (Linear 층 입력용): {flat.shape}")
```